



L OVELY
P ROFESSIONAL
U NIVERSITY

Transforming Education Transforming India

INT331 - FUNDAMENTALS OF DEVOPS

PROJECT SYNOPSIS

(StockFlow - DevOps-Enabled Inventory Management System)

Submitted By :

Name : Annamneedi Suresh Kumar Registration No. : 12317960

Section : RKO030B / Group 2 Roll No. : 56

Submitted To : Manpreet Kaur

1. PROJECT TITLE

StockFlow: DevOps-Enabled Inventory Management System with Containerized Architecture, Monitoring, and CI/CD Pipeline

2. INTRODUCTION

Small businesses often track products, suppliers, and stock movement in spreadsheets. That works for a short time, but it becomes unreliable once products increase, multiple people update the same records, or managers need a clear view of what is low in stock. Mistakes in inventory data can lead to delayed restocking, lost sales, excess purchase cost, and poor visibility into day-to-day operations.

StockFlow is designed as a full-stack inventory management system that handles these practical problems through a web application. The system allows users to manage products, categories, suppliers, stock-in and stock-out transactions, low-stock alerts, dashboard summaries, and inventory reports. The backend is built with FastAPI and PostgreSQL, while the frontend is a React-based dashboard served through a Dockerized production setup.

The main purpose of this project is to show how a real application can be built and operated using DevOps practices. The project does not stop at writing application logic. It also covers containerization with Docker, orchestration with Docker Compose, reverse proxy routing with Nginx, automated checks with GitHub Actions, health endpoints, Prometheus metrics, and Grafana dashboards. This makes the project suitable for demonstrating both application development and the operational workflow used in professional software teams.

In StockFlow, every major part of the system runs as a separate service: frontend, backend, PostgreSQL, Nginx, Prometheus, and Grafana. These services can be started locally with one Docker Compose command. The same repository also contains tests, environment configuration, a production-style compose file, monitoring configuration, and documentation. This gives the project a complete DevOps shape while keeping the business domain simple enough to explain clearly.

3. PROBLEM STATEMENT

Application-Level Challenges

- **Manual Inventory Tracking:** Many small businesses still maintain product and stock data in spreadsheets, which makes errors and duplicate records more likely.
- **Low Visibility:** Managers may not know which products are running low until the shortage affects sales or daily operations.
- **Stock Mismatch:** Without a transaction history, it is difficult to explain why stock increased or decreased.
- **Supplier Tracking Issues:** Supplier details and product relationships are often stored separately, making follow-up and reporting slower.
- **Limited Reporting:** Manual records do not provide quick inventory value, low-stock lists, or stock movement summaries.

Development and Deployment Challenges

- **Environment Differences:** Applications may work on one machine and fail on another because dependencies, database settings, or service versions differ.
- **Manual Startup Process:** Starting frontend, backend, database, and monitoring tools separately is slow and error-prone.

- **No Automated Verification:** Without CI checks, broken tests or failed frontend builds may reach the main branch unnoticed.
- **Lack of Monitoring:** Teams need health checks and metrics to understand whether the backend is reachable, how APIs are performing, and where errors occur.

StockFlow addresses these issues by combining inventory management features with a repeatable Docker-based runtime, automated tests, health checks, and monitoring.

4. OBJECTIVES

Application Development Objectives

- Design and develop an inventory management system for products, categories, suppliers, and stock movement.
- Implement JWT-based authentication with admin, manager, and staff roles.
- Create REST APIs for product management, supplier management, stock-in, stock-out, alerts, dashboard data, and reports.
- Automatically update product stock whenever a stock transaction is created.
- Detect low-stock and out-of-stock products using the product minimum stock value.
- Provide CSV export for inventory, low-stock, and stock transaction reports.

DevOps Integration Objectives

- Containerize frontend, backend, database, Nginx, Prometheus, and Grafana services.
- Use Docker Compose to run the full local stack with one command.
- Configure Nginx as a single entry point for the React frontend, FastAPI backend, and Grafana dashboard.
- Expose health and metrics endpoints from the backend for monitoring and verification.
- Use Prometheus to collect backend metrics and Grafana to display API monitoring dashboards.
- Add a GitHub Actions workflow that runs tests, builds the frontend, builds Docker images, and validates Docker Compose configuration.

5. SCOPE OF PROJECT

In-Scope Features

- **User Authentication:** Register, login, and current-user APIs with role-based access control.
- **Product Management:** Add, update, delete, view, search, and filter products.
- **Category Management:** Create and maintain product categories.
- **Supplier Management:** Store supplier contact details and related business information.
- **Stock Transactions:** Record stock-in and stock-out events with quantity, reason, user, and timestamp.
- **Low-Stock Alerts:** Identify products where current stock is less than or equal to minimum stock.
- **Dashboard:** Show inventory summary, stock movement, category distribution, low-stock products, and recent transactions.
- **Reports:** Provide inventory, low-stock, and transaction reports with CSV export.
- **DevOps Status:** Expose health, service status, environment, application version, and monitoring-related data.
- **Monitoring Stack:** Configure Prometheus and Grafana for backend API metrics.
- **Docker Runtime:** Run the whole system locally using Docker Compose.

Out-of-Scope Features

- AI-based demand prediction.
- Payment gateway integration.
- Barcode scanner integration.
- Mobile application development.
- Kubernetes deployment.
- Cloud deployment for this submission version.

6. METHODOLOGY / SYSTEM ARCHITECTURE

System Architecture Overview

StockFlow follows a multi-container architecture. The user accesses the system through Nginx. Nginx routes normal web traffic to the React frontend, API traffic to the FastAPI backend, and monitoring dashboard traffic to Grafana. The backend communicates with PostgreSQL for persistent storage. Prometheus periodically scrapes the backend metrics endpoint, and Grafana uses Prometheus as its data source.

- Frontend Layer: React 18, Vite, TypeScript, Tailwind CSS, React Router, TanStack Query, Recharts, and Zustand.
- Backend Layer: FastAPI application with routers, schemas, database models, services, authentication, and tests.
- Database Layer: PostgreSQL container with persistent Docker volume storage.
- Reverse Proxy Layer: Nginx routes / to frontend, /api to backend, /docs to FastAPI documentation, and /grafana to Grafana.
- Monitoring Layer: Prometheus collects metrics from /api/metrics, and Grafana visualizes request rate, latency, errors, and endpoint usage.

Phase 1: Project Setup and Backend Foundation

- Create repository folders for backend, frontend, Nginx, Prometheus, Grafana, GitHub Actions, and documentation.
- Build a FastAPI application with separate modules for configuration, database connection, models, schemas, authentication, routers, services, and tests.
- Configure SQLAlchemy to connect to PostgreSQL in Docker and SQLite during automated tests.
- Add startup database initialization and seed data for demo users, suppliers, categories, products, stock transactions, and alerts.

Phase 2: Authentication and Role-Based Access

- Implement password hashing and token-based authentication.
- Define three roles: admin, manager, and staff.
- Allow admin and manager users to manage products, categories, suppliers, reports, and alerts.
- Allow staff users to view inventory data and create stock transactions, while blocking management actions.

Phase 3: Inventory APIs

- Create product APIs for listing, filtering, creating, updating, deleting, and reading product details.
- Create category and supplier APIs for basic inventory organization.

- Create stock-in and stock-out APIs that update product stock and create transaction records.
- Prevent stock-out requests that would reduce stock below zero.
- Synchronize low-stock and out-of-stock alerts after stock changes.

Phase 4: Dashboard, Reports, and DevOps APIs

- Create dashboard APIs for summary cards, charts, low-stock lists, and recent stock transactions.
- Create report APIs for inventory, low-stock products, transactions, and CSV export.
- Create /api/health to report backend and database status.
- Create /api/devops/status to expose service status for the frontend DevOps page.
- Expose /api/metrics using prometheus-fastapi-instrumentator.

Phase 5: Frontend Integration

- Build the React dashboard interface with login, dashboard, products, stock transactions, suppliers, reports, and DevOps status pages.
- Connect frontend requests to the FastAPI backend through Axios and TanStack Query.
- Serve the production frontend build from an Nginx-based frontend container.

Phase 6: Docker and Docker Compose

- Create a backend Dockerfile using a Python 3.12 base image.
- Create a frontend Dockerfile with a Node build stage and an Nginx runtime stage.
- Define Docker Compose services for postgres, backend, frontend, nginx, prometheus, and grafana.
- Add health checks for PostgreSQL, backend, and frontend containers.
- Use named volumes for PostgreSQL, Prometheus, and Grafana data.

Phase 7: CI and Monitoring

- Add GitHub Actions to install dependencies, run backend tests, build the frontend, build Docker images, and validate Docker Compose configuration.
- Configure Prometheus to scrape backend metrics from backend:8000/api/metrics.
- Provision Grafana with a Prometheus data source and a StockFlow API monitoring dashboard.

7. TECHNOLOGY STACK & TOOLS

Category	Technologies
Backend	Python 3.12, FastAPI, SQLAlchemy, Pydantic, Uvicorn
Authentication	JWT-style bearer tokens, password hashing, role-based access control
Frontend	React 18, Vite, TypeScript, Tailwind CSS, React Router
State and Data	TanStack Query, Axios, Zustand
Charts and UI	Recharts, Lucide React, Framer Motion
Database	PostgreSQL, SQLite for tests
Containerization	Docker, Docker Compose
Reverse Proxy	Nginx
Monitoring	Prometheus, Grafana, prometheus-fastapi-instrumentator
Testing	Pytest, HTTPX
CI/CD	GitHub Actions

8. ESSENTIAL DEVOPS COMMANDS & WORKFLOWS

8.1 Docker Commands for Containerization

Build backend Docker image:

```
docker build -t stockflow-backend:v1.0 ./backend
docker build -t stockflow-backend:latest ./backend
```

Build frontend Docker image:

```
docker build -t stockflow-frontend:v1.0 ./frontend
docker build -t stockflow-frontend:latest ./frontend
```

View running containers and logs:

```
docker ps
docker logs -f stockflow-backend-1
```

8.2 Docker Compose Commands

Start all services:

```
docker compose up --build
docker compose up --build -d
```

Stop services:

```
docker compose down
```

View logs and service status:

```
docker compose logs -f
docker compose ps
```

Validate compose configuration:

```
docker compose config
```

8.3 Backend Commands

Run backend tests:

```
cd backend
python -m pytest
```

Start the backend locally without Docker:

```
cd backend
uvicorn app.main:app --reload
```

Seed demo data:

```
cd backend
python -m app.seed
```

8.4 Frontend Commands

Install dependencies and start development server:

```
cd frontend
npm install
npm run dev
```

Build production frontend:

```
npm run build
```

8.5 Makefile Commands

Command	Purpose
make up	Start the local stack with Docker Compose.
make down	Stop all containers.
make logs	Follow container logs.
make test	Run backend Pytest suite.
make seed	Seed demo backend data.
make prod	Run the production-style compose file.
make ps	Show service status.

8.6 CI/CD Pipeline Configuration (GitHub Actions)

The GitHub Actions workflow runs on push and pull request events targeting the main branch. It installs backend dependencies, runs tests, installs frontend dependencies, builds the frontend, builds Docker images tagged with the commit SHA, and validates the Docker Compose file.

```
name: StockFlow CI
on: [push, pull_request]
jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install -r backend/requirements.txt
      - run: cd backend && python -m pytest
      - uses: actions/setup-node@v4
      - run: cd frontend && npm ci && npm run build
      - run: docker build -t sf-backend:$GITHUB_SHA ./backend
      - run: docker build -t sf-frontend:$GITHUB_SHA ./frontend
      - run: docker compose config
```

9. EXPECTED OUTCOMES

Application Outcomes

- A working inventory management system with authentication, products, suppliers, categories, stock movement, alerts, dashboard data, and reports.
- Accurate stock updates for stock-in and stock-out transactions.
- Low-stock and out-of-stock detection based on each product minimum stock value.
- CSV export support for useful inventory reports.
- A React dashboard that presents inventory data, charts, tables, and DevOps status in a clean interface.

DevOps Outcomes

- A complete local Docker Compose setup for frontend, backend, PostgreSQL, Nginx, Prometheus, and Grafana.
- A single Nginx entry point for the app, backend API, FastAPI docs, and Grafana.
- Backend health and metrics endpoints that support deployment verification and monitoring.
- Prometheus metrics collection and Grafana dashboard visualization.

- GitHub Actions workflow that checks backend tests, frontend build, Docker image builds, and Compose configuration.
- Clear documentation and repeatable commands for running, testing, and presenting the project.

10. CONCLUSION

StockFlow demonstrates how an inventory management application can be built with both business functionality and DevOps practice in mind. The application covers common inventory tasks such as product management, supplier tracking, stock transactions, low-stock alerts, dashboard summaries, and report exports. These features give the project a practical domain that is easy to test and explain.

The DevOps side gives the project its main strength. Docker keeps the runtime consistent, Docker Compose starts the full stack with one command, and Nginx provides a clean entry point for the frontend, backend, documentation, and Grafana. Prometheus and Grafana make the backend observable by collecting and displaying API metrics. GitHub Actions adds automated verification so that tests, frontend builds, Docker image builds, and Compose validation run before changes are accepted.

For an academic DevOps submission, StockFlow shows more than a basic CRUD application. It shows how a full-stack system can be structured, containerized, monitored, and verified in a way that resembles a production workflow. The project is ready for local demonstration through Docker Compose and can be extended later with cloud deployment, stricter admin controls, PDF exports, barcode support, or advanced analytics.

11. REFERENCES

1. FastAPI Documentation. <https://fastapi.tiangolo.com>
2. React Documentation. <https://react.dev>
3. Docker Documentation. <https://docs.docker.com>
4. Docker Compose Documentation. <https://docs.docker.com/compose>
5. PostgreSQL Documentation. <https://www.postgresql.org/docs>
6. GitHub Actions Documentation. <https://docs.github.com/actions>
7. Prometheus Documentation. <https://prometheus.io/docs>
8. Grafana Documentation. <https://grafana.com/docs>
9. SQLAlchemy Documentation. <https://docs.sqlalchemy.org>